

Define copy-constructibility-from-bytes

Document #: P3960R0
Date: 2026-01-15
Project: Programming Language C++
SG1, Evolution, Library Evolution
Reply-to: Mark Hoemmen
<mhoemmen@nvidia.com>
Ruslan Arutyunyan
<ruslan.arutyunyan@intel.com>

Contents

- 1 Authors
- 2 Abstract
- 3 Motivation
 - 3.1 Accelerators launch parallel algorithms by byte-copying their arguments
 - 3.2 Type trait that models launching a parallel algorithm on an accelerator?
 - 3.3 `transform_view` may not be trivially copyable, even if its components are
 - 3.4 Work-around couples algorithm and view implementations
 - 3.5 Using iterators instead of views doesn't help
 - 3.6 We want to retain *movable-box*'s copyability
- 4 Our solution: "Copy-constructibility-from-bytes"
 - 4.1 Define copy-constructible-from-bytes type property
 - 4.2 Copy-constructible-from-bytes types are implicit-lifetime types
 - 4.3 Copy-construction from bytes
 - 4.4 Adjust `start_lifetime_as` and `bit_cast` wording accordingly
 - 4.5 Proposed name avoids conflict with `is_trivially_copy_constructible`
 - 4.6 *movable-box* is already an implicit-lifetime type
 - 4.7 Examples of types that should vs. should not be affected
 - 4.7.1 "Morally trivially copyable" types
 - 4.7.2 "Thankfully not trivially copyable" types
 - 4.7.3 Proxy reference types

- 4.7.4 Types with external state affecting how they copy
- 5 What we do NOT propose
 - 5.1 Make lambdas assignable if their members are?
 - 5.2 Decree that *movable-box* is trivially copyable if its component is?
 - 5.3 Let types opt into trivial copyability?
 - 5.4 “Trivial move” (that is, trivial relocation)?
- 6 Implementation
 - 6.1 Compiler changes due to Core language relaxation?
 - 6.2 Type trait
- 7 Acknowledgments
- 8 References
- 9 Proposed wording
 - 9.1 Add `__cpp_lib_copy_constructible_from_bytes` feature test macro
 - 9.2 Change beginning of [class.prop]
 - 9.3 Add example to end of [class.prop]
 - 9.4 Change beginning of [basic.types.general]
 - 9.5 Change [obj.lifetime]
 - 9.6 Change [bit.cast]
 - 9.7 Change [meta.unary.prop]
 - 9.8 Change [meta.reflection.traits]
 - 9.9 Change [meta.type.synop]

§ 1 Authors

- Mark Hoemmen (NVIDIA)
- Ruslan Arutyunyan (Intel)

§ 2 Abstract

Given an object `src` of trivially-copyable type `T`, we can copy the object’s value representation to an array of bytes, and implicitly create a new `T` object (e.g., with `start_lifetime_as`) in the array of bytes. The result will hold the same value as `src`.

What if `T` is implicit-lifetime, and meets *almost* all the criteria for a trivially copyable type, but has nontrivial copy and move assignment operators? The `start_lifetime_as` function still implicitly creates a `T` object, but the value of this object is unspecified.

What *should* happen is unambiguous: `T` has a trivial copy constructor, so nothing should happen to the value representation upon implicit object creation. The new object gets a valid value representation and so it's an object that holds the same value as `src`.

C++ developers want this behavior for many applications that need to communicate objects by bitwise copy. Our original motivation is that `ranges::transform_view` and `ranges::zip_transform_view` are not trivially copyable if their function object is a lambda that captures an `int` by value. This hinders parallelization of ranges algorithms on accelerators with separate memories. Other motivations include fast serialization, remote procedure calls, copying C++ objects through code written in other languages, and communicating objects over a network.

We propose making “what should happen” well-defined behavior by introducing a narrowing of trivial copyability called *copy-constructibility-from-bytes* that does not require a trivial copy or move assignment operator. (We do not call this “trivial copy-constructibility” because that would conflict with the existing `is_trivially_copy_constructible` type trait, that has too narrow of a meaning for our purpose.)

Our definition makes all copy-constructible-from-bytes types also implicit-lifetime types, so we can use `start_lifetime_as` to start the lifetime of such an object using a copy of an existing object's value representation.

§ 3 Motivation

§ 3.1 Accelerators launch parallel algorithms by byte-copying their arguments

Standard Library users would like parallel algorithms to execute on accelerators (such as GPUs) if possible. The parallel algorithm generally would be invoked in ordinary C++ code that is executed on a “host” CPU thread, but the algorithm would perform its computations on the accelerator.

Many kinds of parallel accelerators have their own memories. Directly accessing “host” (ordinary C++) memory from the accelerator, or accelerator memory from a host thread, may be either impossible or slow. “Impossible” is not Standard C++, but “slow” is. For example, NUMA (non-uniform memory access) may result in different memory pages having different access latencies to different threads, depending on the physical hardware on which the thread runs. To forestall these performance or correctness issues, C++ code can first copy data from host memory to accelerator memory before executing the algorithm. Depending on the accelerator, copying may be required or just a possible optimization.

Copying from host to accelerator memory happens bitwise, using `memcpy` or a non-Standard function that behaves like it. If the objects to copy can't be correctly copied bitwise, then the only other option is serialization and deserialization. That is, the accelerator would need to run extra code (which users would normally write, at least pre-Reflection) to “deserialize” raw data into objects. Accelerator users generally don't want to incur this overhead. There is

also no Standard serialization interface, so users would have no portable way to make serialization for their types available to implementations.

Algorithms access memory both when accessing the elements of their input and output ranges, and when accessing the views or iterators that represent these ranges. Even if the elements live in accelerator memory, the views themselves might not. Implementations generally address that by copying the views or iterators into accelerator memory before “launching” the parallel operation there. This copying again works byte-wise, as if by `memcpy`. Thus, the same concerns about byte-wise copying apply to views or iterators as to the actual data.

As a result, if an input view, such as a `transform_view` or `zip_transform_view`, cannot be copied byte-wise to the accelerator, then the algorithm’s implementation generally would not attempt to run on the accelerator. It would instead fall back to a host implementation. That would likely be slower, especially if the data to process were already placed in the accelerator’s memory.

§ 3.2 Type trait that models launching a parallel algorithm on an accelerator?

Implementations need a type trait to know whether a parallel algorithms’ arguments can be copied byte-wise to the accelerator. It seems like trivial copyability (`is_trivially_copyable`) is the right trait, but before we accept this, we need to know how launching a parallel algorithm on an accelerator works. A parallel algorithms’ arguments include ranges or iterators, function objects, and possibly “scalars” (e.g., the input value in `ranges::fill`). The ranges or iterators are “metadata” representing actual data – generally collections such as arrays, over which we want to iterate. For correctness and performance, we would like all arguments, and the data to which any metadata refer, in accelerator memory when we launch the parallel algorithm. Suppose that the data are already there, but the metadata are not.

1. Allocate (or have allocated) space for the algorithm’s arguments
2. Byte-wise copy the arguments to accelerator memory
3. Launch the parallel algorithm’s “kernel main” function on the accelerator. The body of this function finds its arguments with the same values as they had on host, and with their lifetimes started as usual.

“Kernel main” refers to an entry point function for the accelerator. In CUDA C++, the dialect of C++ for running on NVIDIA’s GPUs, these are functions marked with the special keyword `__global__` that return `void`.

Step (3) refers to the function’s arguments with two phrases:

- a. “with the same values as they had on host,” and
- b. “with their lifetimes started.”

The type trait best matching the first phrase is trivial copyability. However, adding the second phrase complicates this. Trivial copyability lets us copy bytes between two *existing* objects, but the objects in accelerator memory haven't started their lifetimes yet. We could first default-construct the objects in accelerator memory, but that would require that the objects be both default-constructible and trivially copyable. Those are excessive requirements. We don't *need* to default-construct the objects; we just need them to exist in accelerator memory with the valid object representations that we provide for them. If the objects have nontrivial default constructors, then we have to do extra work.

The phrase “with their lifetimes started” suggests that we should require the arguments to be *implicit-lifetime types*. Such types can have their lifetimes started by one of the following:

- allocation without explicit construction,
- the memcopy that brought over their bytes, or
- `start_lifetime_as` or an implementation-specific analog.

§ 3.3 `transform_view` may not be trivially copyable, even if its components are

The `transform_view` class may not be trivially copyable, even if their underlying view(s) and function object are. In particular, non-mutable lambdas that capture trivially copyable variables by value are trivially copyable, but `transform_view` with such a lambda is not. An example follows (available on [Compiler Explorer](#)). The `zip_transform_view` class has the same issue.

```
#include <ranges>
#include <type_traits>
#include <vector>
// Function object type that acts just like f2 below.
struct F3 {
    int operator() (int x) const {
        return x + y;
    }
    int y = 1;
};
int main() {
    std::vector v{1, 2, 3};
    // operator= is defaulted; lambda type is trivially copyable
    auto f1 = [] (auto x) {
        return x + 1;
    };
    static_assert(std::is_trivially_copyable_v<decltype(f1)>);
    // Capture means that lambda's operator= is deleted,
    // but lambda type is still trivially copyable
    auto f2 = [y = 1] (auto x) {
        return x + y;
    };
    static_assert(std::is_trivially_copyable_v<decltype(f2)>);
    // decltype(view1) is trivially copyable
    auto view1 = v | std::views::transform(f1);
```

```

static_assert(std::is_trivially_copyable_v<decltype(view1)>);
// decltype(view2) is NOT trivially copyable, even though f2 is
auto view2 = v | std::views::transform(f2);
static_assert(!std::is_trivially_copyable_v<decltype(view2)>);
// view3 is trivially copyable, though it behaves just like view2.
F3 f3{};
auto view3 = v | std::views::transform(f3);
static_assert(std::is_trivially_copyable_v<decltype(view3)>);
return 0;
}

```

Both lambdas `f1` and `f2` are trivially copyable, but `std::views::transform(f2)` is *not* trivially copyable. The specification of `transform_view` and `zip_transform_view` permits this. The wording expresses the input function object of type `F` as stored in an exposition-only `movable-box<F>` member. The `f2` lambda has a capture that gives it a `=deleted` copy assignment operator. Nevertheless, `f2` is still trivially copyable, because each of its default copy and move operations is either trivial or deleted, and its destructor is trivial and not deleted.

The problem is `movable-box`. As [\[range.move.wrap\] 1.3](#) explains, since `copyable<decltype(f2)>` is not modeled, `movable-box<decltype(f2)>` provides a nontrivial, not deleted copy assignment operator. This makes `movable-box<decltype(f2)>`, and therefore `transform_view` and `zip_transform_view`, not trivially copyable.

This doesn't feel like an essential limitation. The `f2` lambda is effectively a struct with one `int` member. Why can't I `memcpy` `views::transform(f2)` wherever I need it to go? Even worse, `f3` is a struct just like `f2`, yet `views::transform(f3)` *is* trivially copyable.

§ 3.4 Work-around couples algorithm and view implementations

As we've shown above, `transform_view` or `zip_transform_view` may not be trivially copyable, even if their components are. One way to work around this is to take the view apart, copy the components separately to accelerator memory, and then reassemble a new view of the same type on the accelerator. This would let parallel algorithms work with a `transform_view`, as long as it has a trivially copyable underlying view and function object. This would require specializing the parallel algorithm implementation on different view or iterator types.

This approach has three disadvantages.

1. It adds implementation complexity, especially with deeply nested views.
2. It may increase compilation time, especially with deeply nested views.
3. The Standard views generally do not have a public interface that exposes all their components. As a result, this approach couples the parallel algorithm implementation to the view implementation.

As examples of (3), `transform_view` and `zip_transform_view` do not offer a Standard public interface to their function object. While `transform_view` has a `base()` member function that exposes its underlying view, `zip_transform_view` does not.

Implementations such as `libc++` already accept this coupling in order to optimize some algorithms for some iterator or view types. The Standard presumes that implementers have complete control over the entire implementation and can give any algorithm access to any nonpublic component of a view or iterator. In the extreme case, this leads to a “fully coupled” implementation, where many parallel algorithms have specializations on many different view and iterator types.

We do not advocate a fully coupled approach for four reasons.

1. Software engineering best practice generally prefers minimizing coupling between components.
2. [P2500](#) proposes opening up the Standard algorithms to customization. In order to be portable, customizations would still need to work with Standard view and iterator types.
3. Accelerator vendors would like to provide parallel algorithms that can be used across different C++ implementations. These necessarily live in a different namespace than `std`, but otherwise aim to conform to the Standard. Users would prefer these algorithms to work with Standard view and iterator types.
4. Ranges were designed to be composable with any classes that conform with the Standard’s requirements on views, not just the views that are in the Standard itself. Specializing the algorithms for only Standard views would hinder composability. Users would like to build their ranges pipelines with both Standard and non-Standard views, pass them to Standard parallel algorithms, and have those algorithms run in parallel.

Regarding (1), users of accelerators consider it a software defect for an accelerator-based implementation not to run operations on the accelerator if possible. Thus, we would prefer not to require so much coupling just to make acceleration work for something so common in ranges as a `views::transform`.

Regarding (2) and (3), anyone who wants to provide parallel algorithms currently must reimplement much of Ranges – a large portion of the Standard Library. Users must then opt into this reimplementation, rather than using Standard types.

Regarding (4), if users mix both Standard and non-Standard views in their code, it would create a big discrepancy between what the Standard parallel algorithms actually parallelize and what they do not. Some composition of views may come from a third-party library that users may not be able to control.

Minimizing coupling between algorithms and views calls for making it possible for algorithms to iterate over views without specializing on the view or iterator type. This

includes making views bitwise copyable to accelerators if possible. Doing so would get more parallel algorithms running on accelerators right away.

§ 3.5 Using iterators instead of views doesn't help

The exposition-only members of `transform_view::iterator` and `transform_view::sentinel` suggest that we could specify them to be trivially copyable as long as the underlying view's iterator and sentinel are trivially copyable. In particular, `transform_view::iterator` has an exposition-only pointer to the `transform_view`, and pointers are trivially copyable. However, that doesn't solve the problem for us, because the pointer still points to a `transform_view` that may live in non-accelerator memory. The algorithm was invoked on host, and thus the `transform_view` and its function were created on host. We need a copy of the function, and access to the underlying view's elements, on the accelerator. The easiest way to do that would be to copy the whole `transform_view` from host memory to accelerator memory.

§ 3.6 We want to retain *movable-box*'s copyability

A *movable-box* of a function object with a copy constructor but a deleted copy assignment operator is not trivially copyable because the *movable-box* has a nontrivial copy assignment operator. The *movable-box*'s copy assignment operator ensures that `transform_view` and `zip_transform_view` both model copyable in as many cases as possible. This simplifies both the specification and the implementation of ranges algorithms and views. For example, it avoids special cases for situations like “copy constructible but not copy assignable.” It also makes creation and use of views more forgiving for users. These are useful properties that we want to retain.

§ 4 Our solution: “Copy-constructibility-from-bytes”

§ 4.1 Define copy-constructible-from-bytes type property

All the things we want to do – run parallel algorithms on an accelerator, communicate objects across a network, or serialize objects to disk – naturally want to “copy-construct” a new object from bytes copied from an existing object's value representation. We don't need copy assignment, yet we find ourselves blocked by trivial copyability's requirement that copy assignment (if it exists) be trivial. What we actually need here is not trivial copyability, but “trivial copy-constructibility.” Unfortunately, that name is taken already (see discussion in a section below), so we use the term “copy-constructible-from-bytes.”

We define a *copy-constructible-from-bytes class* (analogously to [\[class.prop\] 1](#)) to be any class

- that has at least one eligible copy constructor,
- where each eligible copy constructor is trivial,

- where each eligible move constructor, if present, is trivial, and
- that has a trivial, non-deleted destructor.

We define *copy-constructible-from-bytes types* to be scalar types, copy-constructible-from-bytes class types, array of such types, or cv-qualified versions of these types. Trivially constructible types are also copy-constructible-from-bytes, but not necessarily the other way around, as our *movable-box* example shows.

§ 4.2 Copy-constructible-from-bytes types are implicit-lifetime types

It turns out that copy-constructible-from-bytes types are also implicit-lifetime types. This means that we can use them with `start_lifetime_as`.

§ 4.3 Copy-construction from bytes

We say that copy-constructible-from-bytes types can be *copy-constructed from bytes*. This means implicitly creating objects from a value representation copied from an existing object, as in the following example.

```
void* dst_mem = std::malloc(sizeof(T));
std::memcpy(dst_mem, &src, sizeof(T));
T* dst_ptr = std::start_lifetime_as<T>(dst_mem);
```

Analogously to [basic.types.general] 3, given an object `src` of copy-constructible-from-bytes type `T`, and given correctly aligned storage for an object of type `T` at address `dst_mem`, copying the bytes of `src` into `dst_mem` and then implicitly creating a `T` object at address `dst_mem` would ensure that `dst_mem` points to a valid `T` object that holds the same value as `src`.

Implicit object creation via `start_lifetime_as` expresses exactly what we want to express here. Other existing syntax options would not. Default initialization with placement new would invoke a possibly nontrivial default constructor. We don't want to use the default constructor or even require that types be default constructible. None of the `uninitialized_*` or `ranges::uninitialized_*` algorithms in `<memory>` would accomplish this.

§ 4.4 Adjust `start_lifetime_as` and `bit_cast` wording accordingly

The `start_lifetime_as` function implicitly creates an object of any complete, implicit-lifetime type. However, the *value* of the created object is only determined for trivially copyable types. Otherwise, its value is unspecified and can be indeterminate. The current specification of `start_lifetime_as` looks like this.

The value of each created object *o* of trivially copyable type ([basic.types.general]) *U* is determined in the same manner as for a call to `bit_cast<U>(E)` ([bit.cast]), where *E* is an lvalue of type *U* denoting *o*, except

that the storage is not accessed. The value of any other created object is unspecified.

It refers to `bit_cast`. The *Returns clause of `[bit.cast]`* has a detailed description of the desired behavior. We would like to preserve this, as it would seem to apply even for copy-constructible-from-bytes types. All we need to do is relax `bit_cast`'s *Constraints*, which currently require that both the input type `To` and the return type `From` are trivially copyable.

This introduces a new question: What if `To` and `From` differ? We haven't talked about that case above. The *Returns clause of `[bit.cast]`* makes this question not matter, as long as the result `To` gets a valid value representation. The `bit_cast` function already permits those type conversions and doesn't protect callers from the consequences of `To` getting an invalid value representation.

For the result and each object created within it, if there is no value of the object's type corresponding to the value representation produced, the behavior is undefined.

Thus, we think it is acceptable to relax the *Constraints* on both `From` and `To`.

We propose two changes.

1. In the *Effects clause of `start_lifetime_as`*, relax “trivially copyable type” to “copy-constructible-from-bytes type”
2. In the *Constraints clause of `bit_cast`*, relax `is_trivially_copyable_v<To>` to `is_copy_constructible_from_bytes_v<To>`, and relax `is_trivially_copyable_v<From>` to `is_copy_constructible_from_bytes_v<From>`.

The Standard specifies `start_lifetime_as_array` using `start_lifetime_as`, so we do not need to change the wording of `start_lifetime_as_array`.

§ 4.5 Proposed name avoids conflict with `is_trivially_copy_constructible`

We would have preferred to call this property “trivial copy-constructibility,” as it is a restriction of trivial copyability for types that are copy constructible but not copy assignable. Unfortunately, this would collide with the existing trait `is_trivially_copy_constructible`, which has a much narrower meaning than we need. If `is_trivially_copy_constructible_v<T>` is true, then this only means that the copy constructor is trivial ([\[meta.unary.prop\]](#)). However, if `is_trivially_copyable_v<T>` is true, then `T` is trivially copyable, which imposes requirements on other special member functions, including the destructor. Our new property needs to impose analogous requirements beyond those implied by `is_trivially_copy_constructible_v<T>` being true.

§ 4.6 `movable_box` is already an implicit-lifetime type

We depend on both the ability to copy algorithm arguments bitwise, and on the ability to create those arguments implicitly from their value representations. The latter requires that the arguments be implicit-lifetime types ([class.prop]).

The Standard specifies *movable-box* as an enumerated list of differences from `optional<T>` ([range.move.wrap]). None of these differences include the destructor. The specification of `optional<T>` says that its destructor is trivial if `is_trivially_destructible_v<T>` is true ([optional.dtor]). Thus, the same applies to *movable-box<F>*. This makes *movable-box<F>* an implicit-lifetime type as long as *F* is.

Lambdas that capture values have a trivial destructor as long as all their members are trivially destructible. This means that we don't need to change the specification of *movable-box* itself in order for our example to work.

§ 4.7 Examples of types that should vs. should not be affected

In order to understand the effects of our proposed change, we present three categories of types with nontrivial copy assignment operators, and discuss whether we would want to permit their copy construction from bytes.

1. “Morally trivially copyable”: Types with a trivial copy constructor and a nontrivial copy assignment operator, where the latter behaves just like a trivial copy assignment operator would behave. These types motivate this proposal; we define `copy-constructible-from-bytes` to include these types.
2. “Thankfully not trivially copyable”: Types that either forbid copying altogether, or where both copy construction and copy assignment are nontrivial and have desired side effects. Such types would not be copy constructible from bytes.
3. Proxy references: Types that are either already trivially copyable, or that have a trivial copy constructor and a nontrivial copy assignment operator. Copy construction from bytes might cause dangling references or pointers, but would introduce no more possibility of dangling than a struct holding a reference or pointer (which is already trivially copyable). Thus, our proposal would not make their behavior worse.
4. Types with external state affecting how they copy: These types might already be trivially copyable. Our proposal would not make their behavior worse.

We don't presume that these categories cover all possible types. However, this analysis increases our confidence that `copy-constructibility-from-bytes` would not make C++ less safe.

§ 4.7.1 “Morally trivially copyable” types

```
int x = 42;
movable-box m{[x] (int y) { return x + y; }};
```

A *movable-box* of a lambda with an `int` value capture is not trivially copy or move assignable, but this is purely for syntactic reasons. Its nontrivial copy resp. move assignment operators just invoke the stored object's trivial copy resp. move constructor. In this case, though, one could argue that the lambda “should” be copy and move assignable, and that the intended meaning of copy and move assignment is unambiguous.

```
struct S {  
    const int x;  
};  
  
int x = 42;  
movable-box m{S{x}};
```

We might then consider a struct with a `const int` member. This is also not copy assignable, so *movable-box* of that struct is also not trivially copyable. In this case, the `const int` declares the intent that `S` (and therefore *movable-box*<`S`>) is not copy assignable. The “nonessential reason” here is again syntactic. The intended value of `bit_cast<S>(x)` is unambiguous, even though the current specification makes it unspecified.

```
class UnfortunateUserChoice {  
    UnfortunateUserChoice(const UnfortunateUserChoice& rhs)  
        = default;  
  
    UnfortunateUserChoice&  
    operator=(const UnfortunateUserChoice& rhs) {  
        if (this != &rhs) {  
            this->x = rhs.x;  
        }  
        return *this;  
    }  
  
private:  
    int x;  
};
```

We might then consider a class with a defaulted copy constructor but a user-provided copy assignment operator. This class might have come about from a C++98 code base that was later upgraded. The copy constructor's `=default` might have been added at that point, but the developer who did that might have forgotten to add `=default` to the copy assignment operator. Like *movable-box*, copy assignment is “accidentally nontrivial.”

§ 4.7.2 “Thankfully not trivially copyable” types

It might help to contrast the above examples with some types that *are* not and *should* not be copy constructible from bytes. These types have “essentially nontrivial” copy assignment, move assignment, and/or destructors. We can think of at least three subcategories.

1. Containers that use dynamic allocation, like `std::vector`
2. Smart pointers that either forbid copying, like `std::unique_ptr`, or that express shared ownership with reference counting, like `std::shared_ptr`

3. Scope guards, where the destructor and possibly also the constructor have a desired side effect. These types tend to forbid copying altogether.

§ 4.7.3 Proxy reference types

Proxy reference types may have a trivial copy constructor, but their copy assignment operator has a desired side effect. For example, `std::vector<bool>::reference` has a defaulted copy constructor that could reasonably be made trivial, for example if reference stores an address of a byte or word in the `std::vector`'s storage. However, its copy assignment operator cannot and should not be trivial, because it copies a bit, not the address.

Copy-construction-from-bytes might be technically correct as long as the reference's `std::vector` object still exists. However, the common applications of copy-construction-from-bytes may copy the source's object representation “far” from the source object's owning scope. The source object might never have existed in that program, for example when copying over a network or loading from disk.

Expression templates are a special case of proxy reference types. These normally hold references to other objects, which may themselves be expressions. The whole tree of expressions gets evaluated in a container's copy assignment operator, or on conversion to an actual value. Here is a short example (available at this [Compiler Explorer link](#)).

```
#include <cassert>
#include <type_traits>

template<class Left, class Right>
struct Plus {
    constexpr operator float() const {
        return float(left) * float(right);
    }

    const Left& left;
    const Right& right;
};

template<class Left, class Right>
constexpr auto plus(const Left& left, const Right& right) {
    return Plus<Left, Right>(left, right);
}

template<class Left, class Right>
struct Times {
    constexpr operator float() const {
        return float(left) * float(right);
    }

    const Left& left;
    const Right& right;
};

template<class Left, class Right>
constexpr auto times(const Left& left, const Right& right) {
    return Times<Left, Right>(left, right);
}
```

```

}

// A "container" type, into which we assign
// the result of the expression.
struct S {
    float value;

    constexpr operator float() const {
        return value;
    }

    template<class T>
    constexpr S(const T& t) : value(float(t)) {}

    template<class T>
    constexpr float& operator=(const T& t) {
        value = float(t);
        return value;
    }
};

int main() {
    static_assert(std::is_trivially_copyable_v<S>);
    static_assert(std::is_copy_constructible_v<S>);
    static_assert(std::is_copy_assignable_v<S>);

    static_assert(std::is_trivially_copyable_v<Plus<S, S>>);
    static_assert(std::is_copy_constructible_v<Plus<S, S>>);
    static_assert(! std::is_copy_assignable_v<Plus<S, S>>);

    S s1{1.0f};
    S s2{2.0f};
    S s3{4.0f};

    S s4 = times(plus(s1, s2), s3);
    assert(s4.value == 12.0f);

    return 0;
}

```

Note that `Plus` and `Times` are trivially copyable, but not copy assignable. Deleting the copy constructor and copy assignment operator of `Plus` and `Times` makes them not copy constructible, but they remain trivially copyable. Thus, C++ already has the issue that these types are trivially copyable, even though perhaps they shouldn't be.

All these types have in common that they behave like a struct holding a reference or a pointer. Such a struct is trivially copyable. Thus, introducing copy-constructibility-from-bytes would not introduce any more possibility for dangling than these types already have.

§ 4.7.4 Types with external state affecting how they copy

Suppose that a class `fp256` represents a 256-bit floating-point number with 1 sign bit, $p - 1$ significand bits, and $w = 254 - p$ exponent bits. The class stores the bits in a `std::array<uint64_t, 4>` and has no other non-static member data, so it can perfectly be

made trivially copyable. However, users can set a global integer to determine the number of significand bits. This lets them trade between precision and representable range.

(We came up with this example by recalling the ARPREC library (Bailey et al. 2002), though its `mp_real` class performs dynamic allocation and thus could not be trivially copyable.)

Suppose that one program sets the number of significand bits to 236 (the number used by IEEE 754's binary256 type), creates an `fp256` object `src`, copies its object representation, and sends the object representation over a network to another program. That program had already set the number of significand bits to some other value. The receiving program then uses `bit_cast<fp256>` to create an `fp256` object implicitly with the received bytes. The resulting value is a valid `fp256` value, but does not represent the value in the sending program.

Our reading of the *Returns* clause of `[bit.cast]` is that this falls under the the following case:

If there are multiple such values [corresponding to the value representation produced], which value is produced is unspecified.

That is, a single value representation (the 256 bits) can represent multiple floating-point values, and which value it represents depends on external state (the global number of significand bits).

We could imagine adding more global state that controls the total number of bits used for the representation, and establish an invariant that all unused bits must be set to zero. This would fall into the following `[bit.cast]` *Returns* case.

For the result and each object created within it, if there is no value of the object's type corresponding to the value representation produced, the behavior is undefined.

This class is trivially copyable! This proposal would do nothing to affect classes like this. They already behave strangely. C++ currently has no way to tell users not to take the liberties that trivial copyability offers them.

§ 5 What we do NOT propose

This section covers alternate solutions that we do NOT propose here.

§ 5.1 Make lambdas assignable if their members are?

Lambdas with capture clauses currently have a defaulted copy constructor, but deleted assignment operators. This makes *movable-box* of such a lambda have nontrivial assignment operators, and thus makes it not trivially copyable.

We could instead make lambdas have defaulted assignment operators. This would make lambdas trivially assignable (and thus trivially copyable) if their members are.

We like this approach, because it would make lambdas more consistent with other objects of class type. However, we think it should be pursued as a separate proposal. This proposal already exists as [P3963R0](#).

There are other applications for relaxing trivial copyability that do not involve lambdas. A change to lambdas' set of special member functions would require compiler changes, while a minor relaxation of trivial copyability might not.

§ 5.2 Decree that *movable-box* is trivially copyable if its component is?

Launching a parallel algorithm with a `transform_view` on an accelerator does not actually require invoking `transform_view`'s copy assignment operator. It's a purely syntactic requirement for trivial copyability. We could simply *decree* that *movable-box*<F> is trivially copyable if F is.

We do not favor this approach, for three reasons.

1. It would require adding a special case to the core language.
2. The special case would be for an exposition-only type with no Standard name.
 - a. This would hinder use of different Standard Library implementations with the same compiler.
 - b. Users or accelerator vendors would not be able to use this type in portable implementations of their non-Standard views.
3. The benefit of this change would not scale. Users or accelerator vendors whose non-Standard views use an implementation mechanism similar to *movable-box* would not be able to take advantage of the special case.

§ 5.3 Let types opt into trivial copyability?

What if a class could “opt into” trivial copyability, even if it has a nontrivial copy assignment operator? The class C would use some syntax to promise the compiler that any nontrivial special members behave as if they are trivial. In return, `is_trivially_copyable_v<C>` would be true, and both the compiler and users would have permission to make any code transformations that they is allowed to make for trivially copyable types, including the permissions expressed in [\[basic.types.general\]](#) 2-3.

The syntax might look like the special identifier `trivially_relocatable_if_eligible` that was part of C++26's trivial relocation proposal [P2786R13](#). One could imagine an analogous `trivially_copyable_if_eligible` keyword that would let a class opt into the privileges of trivial copyability. Other prior art includes [Clang's `trivial\_abi` attribute](#).

We do not propose this approach here, because trivial copyability is more coarse-grained than what our applications actually need. We do not need to copy bytes between existing

objects, so we do not need to bypass a nontrivial copy assignment operator. While we could imagine narrowing his approach to specific member functions (“I promise that this copy assignment operator behaves as if it is trivial”), we could not come up with use cases to justify the risk of letting users make these kinds of promises.

§ 5.4 “Trivial move” (that is, trivial relocation)?

Suppose that we could “trivially move” a `transform_view` from host to accelerator memory, instead of trivially copying it. By “trivially move,” we mean

1. ending the lifetime of the `transform_view` object in host memory while retaining its storage;
2. copying the storage bytes from host to accelerator memory, after which point the host storage is no longer needed and can be freed; and
3. implicitly creating a new `transform_view` object in accelerator memory using the bytes copied in (2).

Another way to say “trivial move” is “trivial relocation.” The Standard doesn’t currently have a way to express this. C++26’s trivial relocation proposal [P2786](#) did not survive National Body comment challenges and was removed at the Kona meeting in November 2025.

We do not favor this approach for two reasons.

1. It would not solve our *movable-box* problem without further changes, because *movable-box* of a lambda with captures also has a nontrivial move assignment operator.
2. We want to copy objects, not move them. We don’t need or necessarily even want to end the lifetime of the source objects.

§ 6 Implementation

§ 6.1 Compiler changes due to Core language relaxation?

It’s not clear what in the compiler would *need* to change in order to make this work.

§ 6.2 Type trait

Here is a possible implementation of the new type trait.

```
template <typename T>
constexpr bool
is_copy_constructible_from_bytes_v =
    is_copy_constructible_v<T> && // at least one eligible copy constructor
    // if there is an eligible copy constructor, it is trivial
```

```
(std::is_copy_constructible_v<T> ==
    std::is_trivially_copy_constructible_v<T>) &&
// if there is an eligible move constructor, it is trivial
(std::is_move_constructible_v<T> ==
    std::is_trivially_move_constructible_v<T>) &&
std::is_trivially_destructible_v<T>; // is trivially destructible
```

§ 7 Acknowledgments

Author Mark Hoemmen thanks his NVIDIA colleagues Ilya Burylov, Christof Meerwald, and Gonzalo Brito for reviewing this paper and suggesting wording changes!

§ 8 References

- David H. Bailey, Yozo Hida, Xiaoye S. Li, and Brandon Thompson. “ARPREC: An Arbitrary Precision Computational Package,” Lawrence Berkeley National Laboratory technical report, 2002-10-14. [Available online](#) (last viewed 2025-12-11). Please see also [David H. Bailey’s web page](#) and the [ARPREC GitHub page](#).

§ 9 Proposed wording

Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording.

§ 9.1 Add `__cpp_lib_copy_constructible_from_bytes` feature test macro

In [version.syn], add the `__cpp_lib_copy_constructible_from_bytes` macro by replacing YYYYMM below with the integer literal encoding the appropriate year (YYYY) and month (MM).

```
#define __cpp_lib_transparent_operators 201510L // freestanding, also in
    <memory>, <functional>
```

```
#define __cpp_lib_copy_constructible_from_bytes YYYYMM // freestanding,
    also in <memory>, <type_traits>
```

```
#define __cpp_lib_tuple_element_t 201402L // @_freestanding, also in_2
    <tuple>
```

§ 9.2 Change beginning of [class.prop]

Change the beginning of [class.prop] (“Properties of classes”) as follows.

¹ *A copy-constructible-from-bytes class is a class:*

(1.1) — that has at least one eligible copy constructor ([special], [class.copy.ctor]),

(1.2) — where each eligible copy constructor is trivial,

(1.3) — where each eligible move constructor, if present, is trivial, and

(1.4) — that has a trivial, non-deleted destructor ([class.dtor]).

[Editor's note: We say “at least one eligible copy constructor” because a class *X* could have multiple copy constructors *X*(const *X*&) and *X*(*X*&, int=1), for example, per [class.copy.ctor] 1. These are not “of the same kind” per [special] 5, and thus both of them could be eligible, if invoked with a nonconst *X* object.]

2 A *trivially copyable class* is a class:

(2.1) — that has at least one eligible copy constructor, move constructor, copy assignment operator, or move assignment operator ([special], [class.copy.ctor], [class.copy.assign]),

(2.2) — where each eligible copy constructor, move constructor, copy assignment operator, and move assignment operator is trivial, and

(2.3) — that has a trivial, non-deleted destructor ([class.dtor]).

3 A class *S* is a *standard-layout class* if it:

§ 9.3 Add example to end of [class.prop]

Add an example to the end of [class.prop] (“Properties of classes”) as follows.

[Note 3: Aggregates of class type are described in [dcl.init.aggr]. — *end note*]

16 A class *S* is an *implicit-lifetime class* if

(16.1) — it is an aggregate whose destructor is not user-provided or

(16.2) — it has at least one trivial eligible constructor and a trivial, non-deleted destructor.

[Note 4: A copy-constructible-from-bytes class *S* is also an implicit-lifetime class. — *end note*]

[Example 3: If *S* is a copy-constructible-from-bytes class and *src* is an object of type *S*, then the following starts the lifetime of an *S* object at *dst_ptr* with the same object representation as that of *src*.

```
auto* dst_mem = static_cast<std::byte*>(
    std::aligned_alloc(alignof(S), sizeof(S)));
std::memcpy(dst_mem, &src, sizeof(S));
S* dst_ptr = std::start_lifetime_as<S>(dst_mem);
```

— *end example*]

§ 9.4 Change beginning of [basic.types.general]

Change the beginning of [basic.types.general] as follows.

- 1 [Note 1: [basic.types] and the subclauses thereof impose requirements on implementations regarding the representation of types. There are two kinds of types: fundamental types and compound types. Types describe objects, references, or functions. — *end note*]
- 2 For any object `src` (other than a potentially-overlapping subobject) of copy-constructible-from-bytes type `T` ([class.prop]), the underlying bytes ([intro.memory]) making up `src` can be copied into an array of unsigned `char` or `std::byte` ([cstddef.syn]). If the start of the array has alignment at least to `alignof(T)` and an object `dst` of type `T` is implicitly created within the region of storage occupied by the array ([intro.object]), then `dst` shall subsequently hold the same value as `src`.

[Example 1: At the end of this example, `dst_ptr` points to a valid `T` object that holds the same value as `src`.

```
T src(valid_T_object());
auto* dst_mem = static_cast<std::byte*>(
    std::aligned_alloc(alignof(T), sizeof(T)));
std::memcpy(dst_mem, &src, sizeof(T));
T* dst_ptr = std::start_lifetime_as<T>(dst_mem);
```

— *end example*

[Editor's note: Adoption of the proposed resolution of CWG 2489 at the February 2023 meeting removed `char` from the list of types in [intro.object] 3. This is why the list of copy destination types for copy-constructible-from-bytes types differs from the list for trivially copyable types in the following paragraph.]

- 3 For any object (other than a potentially-overlapping subobject) of trivially copyable type `T`, whether or not the object holds a valid value of type `T`, the underlying bytes ([intro.memory]) making up the object can be copied into an array of `char`, unsigned `char`, or `std::byte` ([cstddef.syn]). [Editor's note: We omit the existing footnote link because we do not know how to format it.] If the content of that array is copied back into the object, the object shall subsequently hold its original value.

[Example 2:

```
constexpr std::size_t N = sizeof(T);
char buf[N];
T obj; // obj initialized to its original value
std::memcpy(buf, &obj, N); // between these two calls to std::memcpy,
    obj might be modified
std::memcpy(&obj, buf, N); // at this point, each subobject of obj of
    scalar type holds its original value
```

— *end example*

§ 9.5 Change [obj.lifetime]

Change [obj.lifetime] (“Explicit lifetime management”) as follows.

```
template<class T>
T* start_lifetime_as(void* p) noexcept;
```

```
template<class T>
    const T* start_lifetime_as(const void* p) noexcept;
template<class T>
    volatile T* start_lifetime_as(volatile void* p) noexcept;
template<class T>
    const volatile T* start_lifetime_as(const volatile void* p) noexcept;
```

- 1 *Mandates:* T is an implicit-lifetime type ([basic.types.general]) and not an incomplete type ([basic.types.general]).
- 2 *Preconditions:* [p, (char*)p + sizeof(T)) denotes a region of allocated storage that is a subset of the region of storage reachable through ([basic.compound]) p and suitably aligned for the type T.
- 3 *Effects:* Implicitly creates objects ([intro.object]) within the denoted region consisting of an object *a* of type T whose address is p, and objects nested within *a*, as follows: The object representation of *a* is the contents of the storage prior to the call to start_lifetime_as. The value of each created object *o* of ~~trivially-copyable~~copy-constructible-from-bytes type ([basic.types.general]) U is determined in the same manner as for a call to bit_cast<U>(E) ([bit.cast]), where E is an lvalue of type U denoting *o*, except that the storage is not accessed. The value of any other created object is unspecified.

[Note 1: The unspecified value can be indeterminate. — end note]

- 4 *Returns:* A pointer to the *a* defined in the *Effects* paragraph.

[Editor's note: We do not need to change start_lifetime_as_array, since the Standard specifies it in terms of start_lifetime_as.]

§ 9.6 Change [bit.cast]

Change [bit.cast] (“Function template bit_cast”) as follows.

```
template<class To, class From>
    constexpr To bit_cast(const From& from) noexcept;
```

- 1 *Constraints:*
 - (1.1) — sizeof(To) == sizeof(From) is true;
 - (1.2) — ~~is_trivially_copyable_v~~is copy constructible from bytes v<To> is true.
 - (1.3) — ~~is_trivially_copyable_v~~is copy constructible from bytes v<From> is true.

§ 9.7 Change [meta.unary.prop]

Change [meta.unary.prop] (“Type properties”) as follows.

Template	Condition	Preconditions
...	...	...

template<class T> struct is_const;	T is const-qualified ([basic.type.qualifier])	
template<class T> struct is_volatile;	T is volatile-qualified ([basic.type.qualifier])	
<u>template<class T> struct is_copy_constructible_from_bytes;</u>	<u>T is a copy- constructible-from- bytes type ([basic.types.general]).</u>	<u>remove_all_extents_t<T> shall be a complete type or cv void.</u>
template<class T> struct is_trivially_copyable;	T is a trivially copyable type ([basic.types.general])	remove_all_extents_t<T> shall be a complete type or cv void.
template<class T> struct is_standard_layout;	T is a standard-layout type ([basic.types.general])	remove_all_extents_t<T> shall be a complete type or cv void.
...	...	...

§ 9.8 Change [meta.reflection.traits]

Change [meta.reflection.traits] (“Reflection type traits”) as follows.

```
// associated with [meta.unary.prop], type properties
constexpr bool is_const_type(info type);
constexpr bool is_volatile_type(info type);
```

```
constexpr bool is_copy_constructible_from_bytes_type(info type);
```

```
constexpr bool is_trivially_copyable_type(info type);
constexpr bool is_standard_layout_type(info type);
```

§ 9.9 Change [meta.type.synop]

Change [meta.type.synop] (“Header <type_traits> synopsis”) as follows.

```
// [meta.unary.prop], type properties
template<class T> struct is_const;
template<class T> struct is_volatile;
```

```
template<class T> struct is_copy_constructible_from_bytes;
```

```
template<class T> struct is_trivially_copyable;
template<class T> struct is_standard_layout;
```

[Editor's note: Please skip down to the *_v declarations.]

```
// [meta.unary.prop], type properties
template<class T>
constexpr bool is_const_v = is_const<T>::value;
template<class T>
constexpr bool is_volatile_v = is_volatile<T>::value;
```

```
template<class T>
constexpr bool is_copy_constructible_from_bytes_v =
    is_copy_constructible_from_bytes<T>::value;
```

```
template<class T>
constexpr bool is_trivially_copyable_v =
    is_trivially_copyable<T>::value;
template<class T>
constexpr bool is_standard_layout_v = is_standard_layout<T>::value;
```